

Documento de Inicio — Sistema Integral de Administración para Restaurantes (SIAR)

Oct 4, 2026 · Francisco Javier Restrepo Lenis, Joseph Eliat Orozco Agudelo, Paola Andrea Quiñones Segura.

Objetivo

Diseñar e implementar la versión inicial del Sistema Integral de Administración para Restaurantes (SIAR), una aplicación web para gestionar la operación completa de un restaurante, aplicando los patrones de diseño **Patrón Observer, Patrón Repository, Patrón Singleton, Patrón State**

Problema identificado

Un restaurante coordina en tiempo real el salón, la cocina, la caja, el inventario y la administración; cuando cada área trabaja con registros separados (papel, hojas de cálculo o herramientas aisladas), la operación pierde control, velocidad y trazabilidad.

Esa falta de integración se traduce en problemas concretos del día a día, que el SIAR busca resolver módulo por módulo:

Problema	Consecuencia	Módulo del SIAR
Pedidos tomados y comunicados de forma manual o verbal	Errores, demoras en cocina y sin seguimiento del estado del pedido	Pedidos (RF17–RF22)
Estado de mesas y reservas sin un registro central	Mesas asignadas dos veces y reservas perdidas	Mesas (RF12–RF16)
Menú y disponibilidad desactualizados	Se ofrecen productos que ya no están disponibles	Menú (RF07–RF11)
Inventario desconectado de las ventas	Faltantes, desperdicio y consumo que nadie mide	Control de inventario
Cálculo manual de totales, impuestos y descuentos	Errores de cobro y comprobantes inconsistentes	Facturación (RF28–RF32)
Acceso sin roles ni registro de quién hizo qué	Sin responsabilidades claras ni auditoría	Usuarios y roles (RF01–RF06)
Información dispersa y sin consolidar	Decisiones gerenciales sin datos de ventas, productos ni	Reportes (RF33–RF37) y dashboard

Problema	Consecuencia	Módulo del SIAR
	ocupación	

El problema central es, entonces, la ausencia de un sistema único que integre estas operaciones controle el acceso por rol y entregue información gerencial confiable. Además, para que la solución sea mantenible y verificable, debe construirse con arquitectura en capas, patrones de diseño y pruebas, tal como exige el proyecto.

Justificación

El SIAR se justifica porque integra en una sola aplicación web las operaciones que un restaurante suele llevar por separado, y porque su construcción permite aplicar en un caso real lo que exige la asignatura.

- **Operativa:** centraliza pedidos, mesas, menú, inventario y facturación, lo que reduce errores de registro, acorta el camino entre el pedido y su entrega, y deja trazabilidad de cada operación (RNF06).
- **Gerencial:** los reportes y el dashboard entregan datos de ventas, productos más vendidos, consumo de inventario y ocupación de mesas para decidir con información (RF33–RF37).
- **Académica:** el proyecto permite implementar un prototipo funcional aplicando patrones de diseño, realizar pruebas unitarias, de integración y funcionales, y documentar, desplegar y entregar el software, que son los resultados de aprendizaje de la asignatura.
- **Técnica:** la arquitectura en capas, los patrones State, Strategy, Repository y Observer, el control de acceso por roles, las contraseñas cifradas, el uso de Git y la cobertura mínima de pruebas del 70% dejan una base mantenible y verificable (RNF04, RNF05, RNF07–RNF09).

Objetivos

Objetivo general

Diseñar e implementar la versión inicial del SIAR, una aplicación web responsiva que gestione de forma integral la operación de un restaurante, aplicando los patrones de **Patrón Observer**, **Patrón Repository**, **Patrón Singleton**, **Patrón State**

sobre una arquitectura en capas.

Objetivos específicos

1. Implementar la gestión de usuarios y roles (administrador, cajero, mesero y cocinero), con inicio de sesión, recuperación de contraseña, control de acceso por rol y registro de auditoría (RF01–RF06, RNF04–RNF06).
2. Desarrollar los módulos de menú y mesas para administrar productos, categorías, disponibilidad, estado y reserva de mesas (RF07–RF16).
3. Construir el módulo de pedidos con su ciclo de estados (pendiente, en preparación, listo, entregado y facturado) y el envío a cocina (RF17–RF22).

4. Implementar el control de inventario y la facturación, con cálculo de impuestos, descuentos, registro de pago y emisión de comprobante (RF28–RF32).
5. Generar reportes gerenciales y un dashboard administrativo con ventas, productos más vendidos, consumo de inventario, ocupación de mesas e indicadores financieros (RF33–RF37).
6. Aplicar los patrones State (pedido y mesa), Strategy (impuestos, descuentos y pagos), Repository (acceso a datos) y Observer (eventos del pedido) en la solución y verificarla con pruebas unitarias, de integración y funcionales, con una cobertura mínima del 70% (RNF07, RNF09).
7. Versionar el código con Git, documentar técnicamente el sistema, desplegarlo y entregarlo (RNF08, RNF10).

Alcance

Esta etapa cubre el diseño y la implementación inicial de una aplicación web que gestiona la operación del restaurante a través de siete módulos, usada por cuatro roles: administrador, cajero, mesero y cocinero.

Módulo	Incluye	Requisitos
Usuarios y roles	Registrar, editar y eliminar usuarios; gestionar roles; iniciar sesión y recuperar contraseña	RF01–RF06
Menú	Crear, modificar y eliminar productos; categorizarlos y gestionar su disponibilidad	RF07–RF11
Mesas y reservas	Crear y modificar mesas; consultar su estado; reservar y liberar	RF12–RF16
Pedidos	Crear, modificar y cancelar pedidos; agregar productos; enviarlos a cocina y consultar su estado	RF17–RF22
Inventario	Control de inventario según la descripción general del proyecto	Sin RF en el enunciado
Facturación	Generar factura, calcular impuestos, aplicar descuentos, registrar el pago y emitir comprobante	RF28–RF32
Reportes y	Ventas por periodo, productos más vendidos, consumo de inventario, ocupación de mesas e indicadores financieros;	RF33–RF37

dashboard	dashboard administrativo	
-----------	--------------------------	--

Todos los módulos cumplen los requerimientos no funcionales RNF01–RNF10 (web responsivo, disponibilidad mínima del 95%, respuesta en menos de 3 segundos, contraseñas cifradas, control de acceso por roles, auditoría, arquitectura en capas, Git, cobertura de pruebas del 70% y documentación técnica) y se construyen con los patrones State, Strategy, Repository y Observer.

HISTORIAS DE USUARIO

Resumen del Product Backlog:

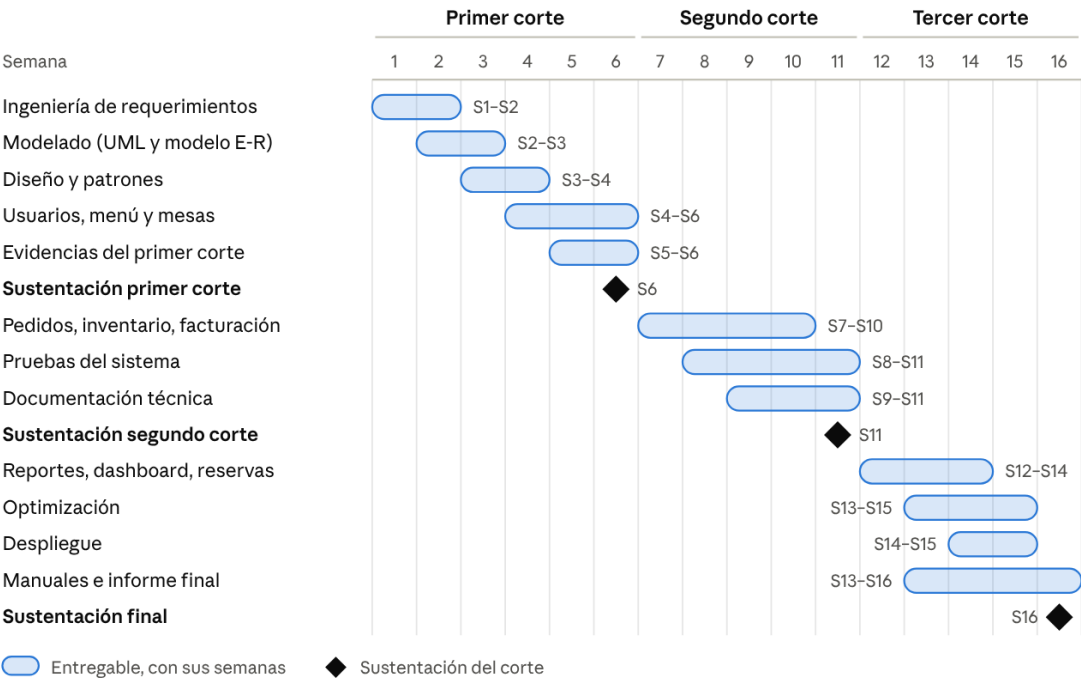
Modulo	Historias	Requisitos	Prioridad
Gestión de usuarios	HU01-HU06	Registro de usuarios	A/M
Gestión de menú	HU07-HU11	Edición de usuarios	A/M
Gestión de mesas	HU12-HU16	Eliminación de usuarios	A/M
Historias técnicas	HT01-HT09	Gestión de roles	

Cronograma

El proyecto se ejecuta en 16 semanas, divididas en tres cortes que cierran cada uno con una sustentación, y lo desarrollan tres personas:

- **Fullstack 1 y Fullstack 2:** base de datos, backend y apoyo en el frontend.
- **Frontend y documentación:** interfaz, historias de usuario, casos de uso y demás documentos.

Tres cortes en 16 semanas, cada uno cierra con una sustentación



cronograma · 16 semanas, 3 cortes, 3 sustentaciones

La distribución semanal dentro de cada corte es una propuesta inicial y se ajusta según el avance real.

Primer corte · semanas 1 a 6 (30%)

Objetivo: diseño e implementación inicial aplicando patrones de diseño.

Entregable	Detalle	Responsable
Ingeniería de requerimientos	Historias de usuario, casos de uso, requerimientos funcionales y no funcionales	Frontend
Modelado	Diagrama de casos de uso, diagrama de clases y modelo entidad-relación	Frontend (UML); Fullstack 1 y 2 (modelo entidad-relación)
Diseño	Arquitectura del sistema, patrones de diseño seleccionados y justificación de cada patrón	Todo el equipo
Desarrollo	Módulos completos de usuarios, menú y mesas	Fullstack 1 y 2 (base de datos y backend); Frontend (interfaz)
Evidencias	Repositorio en GitHub, commits	Todo el equipo

Entregable	Detalle	Responsable
	individuales y video demostración de 10 minutos	
Sustentación	Arquitectura, base de datos, CRUD funcionales y aplicación de patrones	Todo el equipo

Segundo corte · semanas 7 a 11 (30%)

Objetivo: pruebas y consolidación funcional.

Entregable	Detalle	Responsable
Desarrollo	Módulos completos de pedidos, inventario y facturación	Fullstack 1 y 2 (base de datos y backend); Frontend (interfaz)
Pruebas	Pruebas unitarias, de integración y funcionales	Fullstack 1 y 2 (unitarias e integración); Frontend (funcionales)
Evidencias	Matriz de pruebas con caso y resultado para unitarias, integración y funcionales	Frontend
Documentación técnica	Arquitectura y diagramas UML actualizados; manual técnico preliminar	Frontend, con aportes de Fullstack 1 y 2
Sustentación	Demostración completa del flujo Cliente → Pedido → Cocina → Factura	Todo el equipo

Tercer corte · semanas 12 a 16 (40%)

Objetivo: optimización, despliegue y entrega final.

Entregable	Detalle	Responsable
Desarrollo final	Módulos de reportes, dashboard y reservas	Fullstack 1 y 2 (base de datos y backend); Frontend (interfaz)
Optimización	Refactorización, mejora de rendimiento y corrección de hallazgos	Fullstack 1 y 2
Despliegue	Sistema publicado en Internet con URL funcional	Fullstack 1 y 2

Entregable	Detalle	Responsable
Documentación final	Manual técnico (arquitectura, patrones, API y base de datos) y manual de usuario (login, pedidos, inventario, facturación y reportes)	Frontend, con aportes de Fullstack 1 y 2 al manual técnico
Informe final	Introducción, metodología, desarrollo, resultados, conclusiones y trabajo futuro	Todo el equipo
Sustentación final	20 a 30 minutos: problema, solución, arquitectura, patrones utilizados, pruebas ejecutadas, métricas, despliegue y demostración en vivo	Todo el equipo

HISTORIAS DE USUARIO

MODULO 1 : Gestión de usuarios

Convención de prioridades:

A: ALTA

M: MEDIA

B: BAJA

HU01 — Registro de usuarios

Requisito funcional: RF01

Actor: Administrador

Prioridad: A — Alta

Historia de usuario

Como administrador,
quiero registrar nuevos usuarios,
para permitir el acceso al personal autorizado del restaurante.

Criterios de aceptación

1. El sistema debe solicitar nombre, correo electrónico, contraseña y rol al registrar un usuario.
2. El sistema debe validar que el correo electrónico tenga un formato válido.
3. El sistema no debe permitir registrar un usuario utilizando un correo que ya se encuentre registrado.
4. El sistema debe almacenar la contraseña de forma cifrada, de acuerdo con el RNF04.
5. El usuario debe quedar registrado con el rol seleccionado por el administrador.

HU02 — Edición de usuarios

Requisito funcional: RF02

Actor: Administrador

Prioridad: M — Media

Historia de usuario

Como administrador,
quiero editar los datos de un usuario,
para mantener actualizada la información del personal del restaurante.

Criterios de aceptación

1. El sistema debe permitir modificar el nombre del usuario.
2. El sistema debe permitir modificar el correo electrónico.
3. El sistema debe permitir modificar el rol asignado al usuario.
4. El sistema debe validar que el nuevo correo electrónico tenga un formato válido.
5. El sistema debe registrar los cambios realizados en la auditoría, de acuerdo con el RNF06.

quiero eliminar usuarios,
para retirar el acceso al sistema del personal que ya no trabaja en el restaurante.

Criterios de aceptación

1. El sistema debe solicitar una confirmación antes de eliminar un usuario.
2. El sistema no debe permitir que un administrador se elimine a sí mismo.
3. Una vez eliminado, el usuario no debe poder iniciar sesión en el sistema.
4. El sistema debe informar al administrador cuando la eliminación se haya realizado correctamente.

HU04 — Gestión de roles

Requisito funcional: RF04

Actor: Administrador

Prioridad: A — Alta

Historia de usuario

Como administrador,
quiero asignar y gestionar los roles de los usuarios,
para controlar las funciones a las que puede acceder cada persona dentro del restaurante.

Criterios de aceptación

1. El sistema debe permitir asignar un único rol a cada usuario.
2. El sistema debe aceptar únicamente los siguientes roles: administrador, cajero, mesero y cocinero.
3. El sistema debe mostrar a cada usuario únicamente las opciones correspondientes a su rol.
4. El administrador debe poder modificar el rol asignado a un usuario.
5. El control de acceso debe cumplir con las restricciones establecidas en el RNF05.

HU05 — Inicio de sesión

Requisito funcional: RF05

Actor: Usuario

Prioridad: A — Alta

Historia de usuario

Como usuario,
quiero iniciar sesión utilizando mi correo electrónico y contraseña,
para acceder a las funcionalidades correspondientes a mi rol.

Criterios de aceptación

1. El sistema debe solicitar correo electrónico y contraseña para iniciar sesión.
2. Cuando las credenciales sean válidas, el sistema debe permitir el acceso al usuario.
3. Después de iniciar sesión correctamente, el sistema debe mostrar las funcionalidades correspondientes al rol del usuario.

4. Cuando las credenciales sean incorrectas, el sistema debe mostrar un mensaje de error.
5. El sistema debe registrar el ingreso del usuario en la auditoría, de acuerdo con el RNF06.

HU06 — Recuperación de contraseña

Requisito funcional: RF06

Actor: Usuario

Prioridad: M — Media

Historia de usuario

Como usuario,
quiero recuperar mi contraseña,
para volver a acceder al sistema cuando haya olvidado mis credenciales.

Criterios de aceptación

1. El sistema debe permitir solicitar la recuperación de contraseña mediante el correo electrónico registrado.
2. El sistema debe enviar un enlace de recuperación al correo asociado con la cuenta.
3. El enlace de recuperación debe tener un tiempo de vigencia limitado.
4. El sistema debe permitir establecer una nueva contraseña mediante el enlace válido.
5. La nueva contraseña debe almacenarse de forma cifrada.
6. Un enlace de recuperación vencido o inválido no debe permitir cambiar la contraseña.

Módulo 2 — Gestión de Menú

HU07 — Crear productos

Requisito funcional: RF07

Actor: Administrador

Prioridad: A — Alta

Historia de usuario

Como administrador,
quiero crear productos,
para conformar el menú del restaurante.

Criterios de aceptación

1. El sistema debe solicitar nombre, descripción, precio y categoría.
2. El precio del producto debe ser mayor que cero.
3. El sistema no debe permitir registrar dos productos con el mismo nombre.
4. El producto debe quedar asociado a una categoría válida.
5. Una vez creado, el producto debe estar disponible para su gestión en el menú.

HU08 — Modificar productos

Requisito funcional: RF08

Actor: Administrador

Prioridad: A — Alta

Historia de usuario

Como administrador,
quiero modificar los datos de los productos,
para mantener actualizados sus precios y descripciones.

Criterios de aceptación

1. El sistema debe permitir modificar nombre, descripción, precio y categoría.
2. El sistema debe validar que el precio sea mayor que cero.
3. El sistema no debe permitir asignar un nombre que ya pertenezca a otro producto.
4. Cada modificación debe quedar registrada en la auditoría.

HU09 — Eliminar productos

Requisito funcional: RF09

Actor: Administrador

Prioridad: M — Media

Historia de usuario

Como administrador,
quiero eliminar productos,
para retirar del menú aquellos que ya no se ofrecen.

Criterios de aceptación

1. El sistema debe solicitar confirmación antes de eliminar un producto.
2. El producto confirmado como eliminado no debe aparecer en el menú.
3. El sistema debe informar que la operación se realizó correctamente.
4. La eliminación debe quedar registrada en la auditoría.

HU10 — Categorizar productos

Requisito funcional: RF10

Actor: Administrador

Prioridad: M — Media

Historia de usuario

Como administrador,
quiero categorizar los productos,
para organizarlos y facilitar su búsqueda dentro del menú.

Criterios de aceptación

1. Cada producto debe pertenecer a una categoría.
2. El sistema debe permitir utilizar las categorías: entradas, platos fuertes, bebidas y postres.
3. El menú debe permitir filtrar los productos por categoría.
4. El producto debe mostrar la categoría a la que pertenece.

HU11 — Gestionar disponibilidad de productos

Requisito funcional: RF11

Actor: Administrador

Prioridad: A — Alta

Historia de usuario

Como administrador,
quiero marcar un producto como disponible o agotado,
para evitar ofrecer productos que no se encuentren disponibles.

Criterios de aceptación

1. El sistema debe permitir cambiar el estado del producto entre disponible y agotado.
2. El cambio de disponibilidad debe reflejarse inmediatamente en el menú.
3. Un producto marcado como agotado no debe poder agregarse a un pedido.
4. El estado actualizado debe ser consultable por los usuarios que gestionan pedidos.

Módulo 3 — Gestión de Mesas

HU12 — Crear mesas

Requisito funcional: RF12

Actor: Administrador

Prioridad: A — Alta

Historia de usuario

Como administrador,
quiero crear mesas,
para representar la distribución del salón del restaurante.

Criterios de aceptación

1. El sistema debe solicitar el número y la capacidad de la mesa.
2. El número de mesa debe ser único.
3. La capacidad debe ser mayor que cero.
4. Una mesa recién creada debe quedar en estado **libre**.

HU13 — Modificar mesas

Requisito funcional: RF13

Actor: Administrador

Prioridad: M — Media

Historia de usuario

Como administrador,
quiero modificar las mesas,
para actualizar su número o capacidad cuando sea necesario.

Criterios de aceptación

1. El sistema debe permitir modificar el número de la mesa.
2. El sistema debe permitir modificar su capacidad.
3. El nuevo número debe ser único.
4. La capacidad debe ser mayor que cero.
5. El sistema debe advertir al administrador si intenta modificar una mesa reservada u ocupada.

HU14 — Consultar estado de mesas

Requisito funcional: RF14

Actor: Mesero

Prioridad: A — Alta

Historia de usuario

Como mesero,
quiero consultar el estado de las mesas,
para asignar clientes sin cometer errores.

Criterios de aceptación

1. El sistema debe mostrar todas las mesas registradas.
2. Cada mesa debe mostrar su estado actual.
3. Los estados disponibles deben ser: **libre, reservada y ocupada**.
4. El sistema debe permitir filtrar las mesas por estado.
5. La información mostrada debe corresponder al estado actual de cada mesa.

HU15 — Reservar mesa

Requisito funcional: RF15

Actor: Mesero

Prioridad: A — Alta

Historia de usuario

Como mesero,
quiero reservar una mesa,
para asignarla a un cliente.

Criterios de aceptación

1. Solo se debe permitir reservar una mesa que se encuentre en estado **libre**.
2. El sistema debe solicitar nombre del cliente, fecha y hora de la reserva.
3. La reserva debe quedar asociada a la mesa seleccionada.
4. Después de confirmar la reserva, la mesa debe cambiar a estado **reservada**.
5. El cambio de estado debe gestionarse mediante el patrón **State**.

HU16 — Liberar mesa

Requisito funcional: RF16

Actor: Mesero

Prioridad: A — Alta

Historia de usuario

Como mesero,
quiero liberar una mesa,
para dejarla disponible nuevamente.

Criterios de aceptación

1. El sistema debe permitir liberar únicamente mesas reservadas u ocupadas.
2. El sistema debe solicitar confirmación antes de liberar la mesa.
3. Una vez confirmada la operación, la mesa debe pasar a estado **libre**.
4. El nuevo estado debe reflejarse inmediatamente en el sistema.
5. El cambio de estado debe gestionarse mediante el patrón **State**.